

Competitive Programming

Section 50 Editorial: Advanced Algorithms Mixed Contest

Competitive Programming Course

2026-08-14

Table of contents

1	Phase 6 coverage set	5
2	E. Pattern Prefix Matches	6
2.1	From many comparisons to one Z-function	6
2.2	Computing the Z-array	6
2.3	Correctness proof	6
2.4	Complexity and pitfalls	7
2.5	Tests that matter	7
2.6	Full C++ solution	7
2.7	Full Python solution	7
3	F. Suffix LCP Queries	8
3.1	Restatement and monotone predicate	8
3.2	Double substring hashing	8
3.3	Correctness argument	8
3.4	Complexity and alternatives	8
3.5	Tests that matter	9
3.6	Full C++ solution	9
3.7	Full Python solution	9
4	G. Domino Placement	10
4.1	Reduction to bipartite matching	10
4.2	Augmenting-path algorithm	10
4.3	Correctness proof	10
4.4	Complexity and implementation notes	11
4.5	Tests that matter	11
4.6	Full C++ solution	11
4.7	Full Python solution	11
5	H. Weighted Tree Distances	13
5.1	Restatement and why rerooting is needed	13
5.2	First pass: root at vertex 1	13
5.3	Reroot transition	13
5.4	Correctness proof	13
5.5	Complexity and pitfalls	14
5.6	Tests that matter	14
5.7	Full C++ solution	14
5.8	Full Python solution	14
6	I. Balanced Subset Difference	15
6.1	Reformulation	15
6.2	Meet-in-the-middle algorithm	15
6.3	Correctness proof	15
6.4	Complexity and implementation notes	16
6.5	Tests that matter	16

6.6	Full C++ solution	16
6.7	Full Python solution	16
7	J. Monge Partition Table	17
7.1	Baseline interval-partition DP	17
7.2	Monotone optima and divide-and-conquer	17
7.3	Correctness proof	17
7.4	Complexity and implementation notes	18
7.5	Tests that matter	18
7.6	Full C++ solution	18
7.7	Full Python solution	18
8	K. Online Line DP	19
8.1	Turning the recurrence into line queries	19
8.2	Li Chao invariant	19
8.3	Correctness proof	19
8.4	Complexity and pitfalls	20
8.5	Tests that matter	20
8.6	Full C++ solution	20
8.7	Full Python solution	20
9	A–D. Cross-phase core	22
10	Editorial overview	23
10.1	Problem-specific proof map	23
10.2	Detailed solution guide	23
11	A. DAG Grundy	24
11.1	Restatement	24
11.2	Algorithm and proof	24
11.3	Complexity	24
11.4	Full C++ solution	24
11.5	Full Python solution	24
12	B. General CRT	25
12.1	Restatement	25
12.2	Algorithm and proof	25
12.3	Complexity	25
12.4	Full C++ solution	25
12.5	Full Python solution	25
13	C. Fast Fibonacci	26
13.1	Restatement	26
13.2	Algorithm and proof	26
13.3	Complexity	26
13.4	Full C++ solution	26
13.5	Full Python solution	26
14	D. Fibonacci Range Sum	27
14.1	Restatement and observation	27
14.2	Algorithm	27
14.3	Correctness proof	27
14.4	Complexity and implementation	27

14.5 Full C++ solution	27
14.6 Full Python solution	28
14.7 Tests that matter	28
15 Testing discussion	29
16 L. Profitable Disjoint Routes	30
16.1 Reduction to min-cost flow	30
16.2 Correctness proof	30
16.3 Complexity and tests	30
16.4 Full solutions	30

1 Phase 6 coverage set

Category	Contest problem	Detailed editorial
String Algorithms I	E. Pattern Prefix Matches	local package
String Algorithms II	F. Suffix LCP Queries	local package
Flows and Matchings	G. Domino Placement	local package
Rerooting and Advanced Tree DP	H. Weighted Tree Distances	local package
Meet-in-the-Middle	I. Balanced Subset Difference	local package
Divide-and-Conquer DP Optimization	J. Monge Partition Table	local package
Convex Hull Trick and Li Chao Tree	K. Online Line DP	local package

The mixed checker runs these seven self-contained packages from this section's **problems** directory. Each contains its own statement, student stubs, reference implementations, fixed tests, and randomized oracle.

2 E. Pattern Prefix Matches

2.1 From many comparisons to one Z-function

At every text position i , we need the longest prefix of `pattern` equal to a prefix of `text[i..]`. Comparing directly at every position is quadratic on inputs such as repeated `a` characters.

Construct:

```
combined = pattern + '#' + text
```

where `#` is absent from the lowercase input alphabet. The Z-value $z[j]$ is the longest prefix of `combined` matching `combined[j..]`. For text position i , index $|\text{pattern}| + 1 + i$ therefore compares exactly the pattern prefix with the text starting at i .

2.2 Computing the Z-array

Maintain the rightmost interval $[l, r]$ already known to equal a prefix. For a new index i :

- if $i > r$, start comparing from zero;
- otherwise reuse $\min(r-i+1, z[i-l])$ known matching characters;
- extend with direct comparisons;
- update $[l, r]$ if this match reaches farther.

The separator ensures a match originating in the text cannot continue through the boundary into the text again. Capping the extracted Z-value by the pattern length makes the intended limit explicit.

2.3 Correctness proof

By definition of the Z-function, the value at combined index $|\text{pattern}|+1+i$ is the longest prefix of `combined` matching from text position i . The prefix of `combined` begins with `pattern`. Because the separator does not occur in either input string, equality cannot pass its position, so the matched characters are exactly a prefix of `pattern`. Therefore the extracted value is precisely the requested match length.

The standard Z-box algorithm computes the same definition: reused characters lie inside a previously proven prefix-equal interval, and the extension loop checks every additional character explicitly. Hence all reported values are correct.

2.4 Complexity and pitfalls

The right boundary advances at most the combined-string length, so total time and memory are $O(|\text{pattern}| + |\text{text}|)$.

Choose a separator outside the alphabet, offset text indices by $|\text{pattern}|+1$, and output one value for every text position—including zeros. A naive restart of comparisons at every position does not meet the bound.

2.5 Tests that matter

```
Pattern longer than text.
No matching first character anywhere.
Text equal to the pattern.
Highly periodic strings such as aaaa and aaaaaa.
Overlapping matches such as pattern aba in abacaba.
Matches ending exactly at the end of text.
```

For small strings, compare characters directly at each text position.

2.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string p,t;cin>>p>>t;string s=p+"#"+t;
```

2.7 Full Python solution

```
import sys
def main():
    p,t=sys.stdin.read().split();s=p+"#"+t;z=[0]*len(s);l=r=0
    for i in range(1,len(s)):
        if i<=r:z[i]=min(r-i+1,z[i-1])
        while i+z[i]<len(s) and s[z[i]]==s[i+z[i]]:z[i]+=1
        if i+z[i]-1>r:l,r=i,i+z[i]-1
    print(" ".join(str(min(len(p),z[len(p)+1+i])) for i in range(len(t))))
if __name__=="__main__":main()
```

3 F. Suffix LCP Queries

3.1 Restatement and monotone predicate

For suffixes starting at i and j , find their longest common prefix. If their first L characters are equal, then every shorter prefix is also equal; if they differ within the first L , every longer prefix differs as well. Prefix equality is therefore a monotone predicate suitable for binary search.

The remaining requirement is an $O(1)$ substring-equality test. Polynomial rolling hashes provide it after linear preprocessing.

3.2 Double substring hashing

For each of two moduli, precompute powers of a base and prefix hashes:

```
H[r] = hash of s[0..r)
hash(l,r) = H[r] - H[l] * base^(r-l) mod modulus
```

Equal substrings have equal hashes. Using two independent large prime moduli makes an accidental collision negligibly likely.

For query (i, j) , the answer is between zero and $n - \max(i, j)$. Binary-search the largest length L for which the two hash pairs of $s[i..i+L)$ and $s[j..j+L)$ agree.

3.3 Correctness argument

Ignoring the negligible possibility of a hash collision, the hash predicate is true exactly when the two length- L substrings are equal. Equality of length L implies equality for every smaller length, so upper-bound binary search returns the greatest true L . That value is the definition of the LCP of the two suffixes.

The search upper bound prevents either substring from passing the end of the string. In particular, querying a suffix against itself correctly returns its full remaining length.

3.4 Complexity and alternatives

Power and prefix tables take $O(n)$ time and memory. Each query performs $O(\log n)$ hash comparisons, each $O(1)$, for total $O(n + q \log n)$ time.

This intended solution is probabilistic. A suffix array with an LCP array and RMQ can answer deterministically, but requires more machinery. Common hash errors are mixing inclusive and half-open endpoints, failing to normalize a negative remainder, or using powers for the wrong substring length.

3.5 Tests that matter

```
i = j                                -> entire remaining suffix
Different first characters            -> 0
One suffix is a prefix of the other
Repeated string aaaaaa              -> long LCPs at nearby positions
banana queries                       -> overlapping periodic suffixes
Positions near n                     -> upper-bound correctness
```

The randomized oracle compares the two suffixes character by character.

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
int main(){ios::sync_with_stdio(false); cin.tie(nullptr); string s; cin>>s; int n=s.size(); const
```

3.7 Full Python solution

```
import sys
def main():
    d=sys.stdin.buffer.read().split(); s=d[0]; n=len(s); q=int(d[1]); mods=(1_000_000_007,1_000_000_009)
    powers=[]; prefix=[]
    for mod in mods:
        p=[1]*(n+1); h=[0]*(n+1)
        for i,c in enumerate(s): p[i+1]=p[i]*base%mod; h[i+1]=(h[i]*base+c)%mod
        powers.append(p); prefix.append(h)
    def equal(a,b,length):
        for mod,p,h in zip(mods,powers,prefix):
            if (h[a+length]-h[a]*p[length])%mod!=(h[b+length]-h[b]*p[length])%mod: return False
        return True
    out=[]; idx=2
    for _ in range(q):
        a,b=int(d[idx])-1,int(d[idx+1])-1; idx+=2; lo,hi=0,n-max(a,b)
        while lo<hi:
            mid=(lo+hi+1)//2
            if equal(a,b,mid): lo=mid
            else: hi=mid-1
        out.append(str(lo))
    print("\n".join(out))
if __name__=="__main__": main()
```

4 G. Domino Placement

4.1 Reduction to bipartite matching

Color grid cells by parity of $\text{row} + \text{column}$. Cardinal neighbors always have opposite parity. Create one graph vertex for each open cell and an edge for each cardinally adjacent pair, placing even cells on the left and odd cells on the right.

A domino covers exactly one such graph edge. Requiring dominoes not to overlap means no two chosen edges may share a vertex—which is precisely a matching. Thus the geometry disappears after the reduction.

4.2 Augmenting-path algorithm

Maintain $\text{match}[v]$, the left cell currently paired with right-side cell v . For every open left cell, search for an augmenting path:

1. Try each adjacent right cell.
2. If it is free, match it immediately.
3. If it is occupied, recursively try to reroute its current left partner.
4. On success, assign the right cell to the current left cell.

Use a fresh visitation stamp for each search so a right vertex is explored at most once during that attempt.

4.3 Correctness proof

There is a bijection between legal domino placements and matchings: every domino maps to its adjacency edge, and non-overlap maps to disjoint endpoints; every matching maps back to non-overlapping adjacent dominoes.

The augmenting-path procedure starts with an empty matching and increases its size by one whenever it finds an alternating path from an unmatched left vertex to an unmatched right vertex. When no such path exists for any left vertex, Berge's lemma says the matching is maximum. Therefore the matching size, and hence the printed number of dominoes, is optimal.

4.4 Complexity and implementation notes

Let V be the number of open cells and $E \leq 4V$ the adjacency edges. The simple Kuhn algorithm runs in $O(VE)$ worst-case time and uses $O(V+E)$ memory, which is sufficient for a 100×100 grid. Hopcroft–Karp would improve the bound to $O(E \sqrt{V})$ if larger grids were required.

Only start searches from one parity; otherwise each grid adjacency is treated from both sides and the matching logic is invalid. Walls create no graph vertex. In Python, recursive augmenting paths require a sufficiently generous recursion limit or an iterative implementation.

4.5 Tests that matter

```
All walls                -> 0
One open cell            -> 0
Two adjacent versus diagonal cells
A corridor of odd length -> floor(length/2)
Disconnected open regions
A board where greedy local pairing blocks a larger placement
```

For tiny grids, enumerate subsets of adjacency edges and keep the largest vertex-disjoint set.

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;cin>>n>>m;vector<string>g(n)
```

4.7 Full Python solution

```
import sys
def main():
    data=sys.stdin.read().split();n,m=map(int,data[:2]);g=data[2:];match=[-1]*(n*m);seen=[0]
    def dfs(cell):
        if seen[cell]==stamp:return False
        seen[cell]=stamp;r,c=divmod(cell,m)
        for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr,nc=r+dr,c+dc
            if 0<=nr<n and 0<=nc<m and g[nr][nc]==".":
                v=nr*m+nc
                if match[v]==-1 or dfs(match[v]):match[v]=cell;return True
        return False
    ans=0
    for r in range(n):
        for c in range(m):
```

```
        if g[r][c]== "." and (r+c)%2==0:  
            stamp+=1;ans+=dfs(r*m+c)  
    print(ans)  
if __name__=="__main__":main()
```

5 H. Weighted Tree Distances

5.1 Restatement and why rerooting is needed

For every possible root r , compute:

```
answer[r] = sum over v of weight[v] * distance(r,v)
```

A BFS or DFS from every root is $O(n^2)$. Neighboring roots have closely related answers, so compute one root directly and transfer its value across each edge.

5.2 First pass: root at vertex 1

An iterative traversal records `parent`, `depth`, and an order. Then:

```
answer[1] = sum(weight[v] * depth[v])
subtree[u] = sum of weights in u's rooted subtree
```

Compute subtree weights in reverse traversal order.

5.3 Reroot transition

Move the root from parent u to child v . Distances to vertices in v 's subtree decrease by one, changing the sum by $-\text{subtree}[v]$. Distances to all other vertices increase by one, changing it by $\text{total_weight} - \text{subtree}[v]$. Therefore:

```
answer[v] = answer[u] + total_weight - 2 * subtree[v]
```

Applying this formula in parent-before-child order obtains every answer.

5.4 Correctness proof

The first answer is correct by the definition of depth from root 1. Reverse-order accumulation gives each subtree's exact total weight because every child's completed sum is added once to its parent.

For an edge from u to child v , the edge separates the tree into exactly two sides. Rerooting at v shortens every path to the v side by one and lengthens every path to the other side by one. The derived formula accounts for the total weight on both sides exactly. By induction along the rooted tree, if $\text{answer}[u]$ is correct then $\text{answer}[v]$ is correct; since the root answer is correct, all answers are correct.

5.5 Complexity and pitfalls

Both traversals examine every edge a constant number of times, giving $O(n)$ time and $O(n)$ memory. Use 64-bit integers for weights, subtree sums, and answers. Iterative traversal avoids recursion overflow at $n = 200000$.

Do not use subtree vertex counts in the formula—the values are weighted. Zero-weight vertices remain structurally present even though they add nothing to a sum.

5.6 Tests that matter

```
One vertex.
Two vertices with unequal weights.
A path, checking both endpoints and a middle root.
A star with a heavy leaf.
All weights zero.
Large weights on a deep path -> overflow check.
```

For small trees, run BFS from every root and evaluate the definition directly.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
int main(){ios::sync_with_stdio(false); cin.tie(nullptr); int n; cin>>n; vector<ll>w(n); for(ll&x
```

5.8 Full Python solution

```
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];w=d[1:n+1];g=[[[]for _ in range(n)
    for _ in range(n-1):u,v=d[idx]-1,d[idx+1]-1;idx+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;depth=[0]*n;order=[0]
    for u in order:
        for v in g[u]:
            if v!=par[u]:par[v]=u;depth[v]=depth[u]+1;order.append(v)
    sub=w[:]
    for v in reversed(order[1:]):sub[par[v]]+=sub[v]
    total=sum(w);ans=[0]*n;ans[0]=sum(w[i]*depth[i] for i in range(n))
    for u in order:
        for v in g[u]:
            if par[v]==u:ans[v]=ans[u]+total-2*sub[v]
    print(" ".join(map(str,ans)))
if __name__=="__main__":main()
```

6 I. Balanced Subset Difference

6.1 Reformulation

Putting a subset into the first group determines the other group. If the total sum is T and the chosen subset sum is S , the absolute group difference is:

$$|S - (T-S)| = |T - 2S|$$

We need a subset sum as close as possible to $T/2$. Enumerating all 2^{40} subsets is too large, but splitting the values into two halves reduces each enumeration to about 2^{20} .

Signed values do not invalidate the reformulation; they only mean subset sums are not naturally ordered until we sort them.

6.2 Meet-in-the-middle algorithm

1. Split the input into left and right halves.
2. Enumerate all subset sums L of the left half and all subset sums R of the right half, including the empty subset.
3. Sort R .
4. For every x in L , seek a y in R making $x+y$ closest to $T/2$. Binary-search the insertion position for that target and test the neighbor on each side.
5. Minimize $|T - 2(x+y)|$.

Every subset of the original array decomposes uniquely into one left subset and one right subset.

6.3 Correctness proof

Let an optimal full subset have half-sums (x,y) . The algorithm enumerates its left sum x . For this fixed x , the objective as a function of y is the absolute distance from a target, so among sorted right sums its minimum occurs at the first value not below the target or its predecessor. The algorithm checks both. It therefore finds a right sum at least as good as the optimal subset's y . Since every checked pair also corresponds to a valid full subset, the global minimum is exactly optimal.

6.4 Complexity and implementation notes

Each half has at most 2^{20} sums. Enumeration is $O(2^{(n/2)})$; sorting and binary searches give $O(2^{(n/2)} \log 2^{(n/2)})$ time and $O(2^{(n/2)})$ memory.

Use 64-bit signed sums. Always include the empty subset because either group may be empty. With negative totals, avoid a careless integer division by two; equivalently binary-search an integer threshold derived from $|T - 2(x+y)|$ and check both adjacent values.

6.5 Tests that matter

```
n = 0 or one value, if permitted by the contract.
An exact balanced split -> 0.
All values positive with no exact split.
Mixed positive and negative values.
All negative values.
An optimum using the empty subset or all values.
Values near  $10^{12}$  -> 64-bit sums.
```

For small n , enumerate every full subset and evaluate $|T-2S|$.

6.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long;
vector<ll>sums(const vector<ll>&a){vector<ll>s{0};for(ll x:a){int n=s.size();for(int i=0;i<n;i++)s[i+1]=s[i]+x;}return s;}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;cin>>n;vector<ll>a(n);for(ll&x:a)cin>>x;vector<ll>s=sums(a);ll best=1e18;for(int i=0;i<n;i++){ll x=a[i];ll y=s[i+1]-x;ll z=s[n]-x-y;ll v=abs(y-z);if(v<best)best=v;}cout<<best<<endl;return 0;}
```

6.7 Full Python solution

```
import bisect,sys
def sums(a):
    out=[0]
    for x in a:out += [v+x for v in out]
    return out
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];a=d[1:];left=sums(a[:n//2]);right=sums(a[n//2:])
    for x in left:
        target=(total-2*x+1)//2;i=bisect.bisect_left(right,target)
        for j in (i-1,i):
            if 0<=j<len(right):best=min(best,abs(total-2*(x+right[j])))
    print(best)
if __name__=="__main__":main()
```

7 J. Monge Partition Table

7.1 Baseline interval-partition DP

Partition positions $1..n$ into exactly k nonempty contiguous groups. Define:

```
dp[g][i] = minimum cost to partition positions 1..i into g groups
```

If the last group is $[p,i]$, the previous $g-1$ groups cover $1..p-1$:

```
dp[g][i] = min over g <= p <= i of
           dp[g-1][p-1] + cost[p][i]
```

The straightforward evaluation is $O(kn^2)$, too slow for the stated bounds.

7.2 Monotone optima and divide-and-conquer

The input promises a Monge interval-cost table. The standard monotonicity theorem for this recurrence then guarantees:

```
opt[g][i] <= opt[g][i+1]
```

where $\text{opt}[g][i]$ is a minimizing start p for the last group.

To compute one DP layer on output interval $[lo,hi]$, evaluate its midpoint mid , searching only candidate splits $[\text{optL},\text{optR}]$. If best minimizes the midpoint, recursively compute:

```
left outputs  [lo,mid-1] with candidates [optL,best]
right outputs [mid+1,hi] with candidates [best,optR]
```

Monotonicity proves that no discarded candidate can be optimal in either half.

7.3 Correctness proof

The baseline recurrence is exact because every valid partition has a unique start p of its last group, and combining an optimal prefix partition with $[p,i]$ constructs a valid candidate.

For the optimized evaluation, assume the recursive call's candidate interval contains the optimum for every output index it owns. The midpoint search finds its true optimum best . Monotonicity places every optimum to the left at or before best , and every optimum to the right at or after best , so the two recursive candidate ranges preserve the assumption. Induction over recursive calls shows every computed DP value equals the baseline recurrence, and hence $\text{dp}[k][n]$ is optimal.

7.4 Complexity and implementation notes

At one recursion depth, the searched candidate ranges have total length $O(n)$; there are $O(\log n)$ depths. Each of k layers therefore takes $O(n \log n)$, for $O(kn \log n)$ time. DP layers use $O(n)$ memory, while the given cost table itself uses $O(n^2)$.

Exactly g nonempty groups means layer g starts at position $i = g$. Initialize only $dp[0][0] = 0$; impossible states stay at infinity. Be precise when translating one-based groups into the zero-based table $cost[p-1][i-1]$.

7.5 Tests that matter

```
k = 1                -> cost[1][n]
k = n                -> sum of singleton costs
Small table checked by  $O(kn^2)$  DP
Several tied optimal splits -> monotonic tie handling
Large costs          -> infinity and overflow safety
```

Random tests should generate a table known to be Monge and compare against the quadratic recurrence, rather than assuming the optimized code is its own oracle.

7.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll INF=(1LL<<62);
int main(){ios::sync_with_stdio(false); cin.tie(nullptr); int n,k; cin>>n>>k; vector<vector<ll>>
```

7.7 Full Python solution

```
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split())); n,k=d[:2]; v=d[2:]; cost=[v[i*n:(i+1)*n]]
    for group in range(1,k+1):
        cur=[inf]*(n+1)
        def solve(lo,hi,optl,opttr):
            if lo>hi: return
            mid=(lo+hi)//2; best=optl
            for j in range(optl,min(opttr,mid-1)+1):
                value=prev[j]+cost[j][mid-1]
                if value<cur[mid]: cur[mid]=value; best=j
            solve(lo,mid-1,optl,best); solve(mid+1,hi,best,opttr)
        solve(group,n,group-1,n-1); prev=cur
    print(prev[n])
if __name__=="__main__": main()
```

8 K. Online Line DP

8.1 Turning the recurrence into line queries

The transition is:

```
dp[i] = fee[i] + min over j < i of
      (dp[j] + slope[j] * x[i])
```

For a fixed earlier index j , the expression depending on $x[i]$ is a line:

```
y = slope[j] * x + dp[j]
```

Thus each step queries the minimum of all previous lines at coordinate $x[i]$, adds $fee[i]$, then inserts the new line from index i . The insertions must be online because line i contains the just-computed $dp[i]$.

No monotonicity is promised for slopes or query coordinates, so a monotone CHT deque is not valid. A Li Chao tree supports arbitrary order.

8.2 Li Chao invariant

Build the tree over the sorted distinct coordinates appearing in x . Each node stores one line that is best at that node's midpoint among lines compared there. When inserting a new line:

1. At the midpoint, keep the better line in the node.
2. The displaced line can beat the stored line on at most one side because two lines cross at most once.
3. Compare at the interval endpoints and recurse only into the side where the displaced line can still win.

A query follows the root-to-leaf path for its compressed coordinate and takes the minimum of every stored line encountered.

8.3 Correctness proof

The insertion rule never discards a line from a region where it might be optimal: after keeping the midpoint winner, the other line can only become better on one side, and it is recursively stored on precisely that side. Inductively, for every inserted line and every coordinate, either the line is stored on the query path or another stored line is no worse there. Therefore a query returns the minimum over all inserted lines.

Before computing `dp[i]`, the inserted lines correspond exactly to indices $j < i$. The query consequently returns the minimum transition term, and adding `fee[i]` gives the recurrence. Inserting line i restores the invariant for the next iteration. By induction, the final `dp[n-1]` is correct.

8.4 Complexity and pitfalls

There are at most n compressed coordinates. Each insertion and query descends $O(\log n)$ levels, for $O(n \log n)$ time and $O(n)$ tree memory.

- Insert line 0 with intercept `dp[0] = 0` before the first query.
- Query before inserting line i ; otherwise the forbidden choice $j = i$ becomes available.
- Coordinate compression changes only the domain, not the actual x used in line evaluation.
- Use 64-bit arithmetic, or wider intermediates if the contract permits products near the type limit.
- Duplicate slopes and duplicate coordinates must be supported.

8.5 Tests that matter

```
n = 1                                -> dp[0] = 0
One transition                        -> direct formula
Unsorted and duplicate x coordinates
Increasing, decreasing, and duplicate slopes
A newly inserted line wins only on one extreme
Negative slopes and coordinates
Large products                       -> integer-width check
```

The small oracle evaluates every earlier j directly in $O(n^2)$.

8.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll INF=(1LL<<62); struct Line{ll m=0,b=INF;ll at;
int main(){ios::sync_with_stdio(false); cin.tie(nullptr); int n; cin>>n; vector<ll>x(n),m(n),fee=
```

8.7 Full Python solution

```
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()));n=d[0];x=d[1:n+1];m=d[n+1:2*n+1];fee=d[2*n+1:]
    def value(line,at):return line[0]*at+line[1]
    def add(line,p,l,r):
```

```

mid=(l+r)//2
if value(tree[p],coords[mid])>value(line,coords[mid]):tree[p],line=line,tree[p]
if l==r:return
if value(tree[p],coords[l])>value(line,coords[l]):add(line,p*2,l,mid)
elif value(tree[p],coords[r])>value(line,coords[r]):add(line,p*2+1,mid+1,r)
def get(p,l,r,at):
    ans=value(tree[p],coords[at])
    if l==r:return ans
    mid=(l+r)//2
    return min(ans,get(p*2,l,mid,at)if at<=mid else get(p*2+1,mid+1,r,at))
dp=[0]*n;add((m[0],0),1,0,len(coords)-1)
for i in range(1,n):
    at=__import__("bisect").bisect_left(coords,x[i]);dp[i]=fee[i]+get(1,0,len(coords)-1,at)
print(dp[-1])
if __name__=="__main__":main()

```

9 A–D. Cross-phase core

The DAG Grundy, generalized CRT, and two Fibonacci problems revisit earlier advanced ideas. They are the first four exercises of the complete twelve- problem contest; G–L remain its progressively harder half.

10 Editorial overview

10.1 Problem-specific proof map

A. DAG Grundy. Reverse topological order computes successor Grundy values before their parent; mex zero/nonzero gives exactly losing/winning states.

B. General CRT. Two congruences intersect iff their residue difference is divisible by the gcd. Extended Euclid solves the reduced equation and gives the unique merged class modulo the lcm.

C. Fast Fibonacci. The doubling identities preserve the adjacent pair $(F(k), F(k+1))$, so binary decomposition of the index returns the requested value in logarithmic time.

10.2 Detailed solution guide

A. DAG Game Grundy. Process vertices in reverse topological order. The Grundy value of a terminal vertex is 0. For a nonterminal vertex, collect the Grundy values of all outgoing neighbors and take the mex, the smallest nonnegative integer not present. A position is winning exactly when its Grundy value is nonzero: move to a child with Grundy 0; from Grundy 0, every move goes to nonzero.

B. Chinese Remainder Merge. To merge $x = a \pmod m$ and $x = b \pmod n$, first check compatibility: $a \% \gcd(m, n)$ must equal $b \% \gcd(m, n)$. If not, there is no solution. If yes, divide by the gcd and use an extended-gcd inverse to solve for the offset. The new modulus is $\text{lcm}(m, n)$, and the merged residue is normalized into $[0, \text{lcm})$.

C. Fast Doubling Fibonacci. Maintain the pair $(F(k), F(k+1))$. The doubling formulas compute $(F(2k), F(2k+1))$; if the next binary bit is 1, advance once to $(F(2k+1), F(2k+2))$. This gives logarithmic time because each bit of n does constant work.

Common pitfalls. For Grundy values, mex is taken over children only, not all smaller numbers. CRT must handle non-coprime moduli. Fast doubling should keep all operations modulo the requested modulus.

This mixed section checks whether the model is recognized quickly.

- A computes Grundy numbers on a DAG in reverse topological order. A start is winning iff its Grundy number is nonzero.
- B merges two congruences with the generalized CRT. The compatibility test is equality modulo $\gcd(m, n)$.
- C uses fast doubling for Fibonacci, returning the pair $(F(n), F(n+1))$.

All three solutions compress a large object into a smaller invariant: mex, one merged congruence, or a recurrence pair.

11 A. DAG Grundy

11.1 Restatement

For each start vertex in a DAG, print whether the first player wins.

11.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

11.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

11.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,q;if(!
```

11.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,q=data[0],data[1],data[2]; idx=3; g=[] for _ in range(n)
    for _ in range(m): g[data[idx]-1].append(data[idx+1]-1); idx+=2
    gr=[0]*n
    for u in range(n-1,-1,-1):
        seen={gr[v] for v in g[u]}; x=0
        while x in seen: x+=1
        gr[u]=x
    out=["WIN" if gr[data[idx+i]-1] else "LOSE" for i in range(q)]
    print("\n".join(out))
if __name__=="__main__": main()
```

12 B. General CRT

12.1 Restatement

For each pair of congruences, print the smallest nonnegative solution and lcm modulus, or NO.

12.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

12.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

12.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; long long eg(long long a,long long b,long long&x,long long&y){if(!b){x=
```

12.5 Full Python solution

```
import sys,math
def eg(a,b):
    if b==0: return (abs(a),1 if a>=0 else -1,0)
    g,x,y=eg(b,a%b); return g,y,x-(a//b)*y
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        a,m,b,n=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; g,x,y=eg(m,n)
        if (b-a)%g: out.append("NO")
        else:
            l=m//g*n; k=((b-a)//g*x)%(n//g); out.append(f"{{(a+m*k)%l}} {{l}}")
    print("\n".join(out))
if __name__=="__main__": main()
```

13 C. Fast Fibonacci

13.1 Restatement

Answer Fibonacci queries modulo $1e9+7$.

13.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

13.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

13.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

13.5 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

14 D. Fibonacci Range Sum

14.1 Restatement and observation

For a huge inclusive interval $[1, r]$, sum Fibonacci values modulo MOD . The blind step is recognizing the telescoping identity:

$$F(0) + F(1) + \dots + F(t) = F(t+2) - 1.$$

Therefore the requested result is:

$$F(r+2) - F(1+1) \bmod \text{MOD}.$$

14.2 Algorithm

Use fast doubling to obtain $(F(n), F(n+1))$ in $O(\log n)$:

$$\begin{aligned} F(2k) &= F(k) * (2F(k+1) - F(k)) \\ F(2k+1) &= F(k)^2 + F(k+1)^2. \end{aligned}$$

Apply it twice per query and normalize the subtraction.

14.3 Correctness proof

The prefix identity follows by induction from $F(i+2) = F(i+1) + F(i)$. Subtracting the prefix through $1-1$ from the prefix through r gives exactly $[1, r]$. Fast doubling identities compute the required Fibonacci values by splitting the index in half, so the printed normalized difference is correct. $\square$

14.4 Complexity and implementation

Each query costs $O(\log r)$ time and $O(\log r)$ recursion stack; values are always reduced modulo 1,000,000,007. Use unsigned 64-bit indices in C++ for the 10^{18} bound and Python integers in Python.

14.5 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std; using ll=long long; const ll MOD=1000000007;
pair<ll,ll> fib(unsigned long long n){
    if (!n) return {0, 1};
    auto [a, b] = fib(n / 2);
    ll c=a*((2*b%MOD-a+MOD)%MOD)%MOD, d=(a*a%MOD+b*b%MOD)%MOD;
    return n&1?make_pair(d, (c+d)%MOD):make_pair(c,d);
}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int q;cin>>q;while(q--){unsigned lon

```

14.6 Full Python solution

```

import sys
MOD=1_000_000_007
def fib(n):
    if n==0:return 0,1
    a,b=fib(n//2); c=a*((2*b-a)%MOD)%MOD; d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n&1 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split())); out=[]
    for i in range(1,len(data),2): out.append(str((fib(data[i+1]+2)[0]-fib(data[i]+1)[0])%MOD))
    print('\n'.join(out))
if __name__=='__main__':main()

```

14.7 Tests that matter

[0,0], [0,1], one-element ranges, and a range with $l > 0$.
 Large endpoints: never iterate through the interval.

15 Testing discussion

16 L. Profitable Disjoint Routes

16.1 Reduction to min-cost flow

Give every original edge capacity 1 and cost equal to the negative of its profit. Send exactly k units of flow from 1 to n . Integral unit-capacity flow decomposes into k edge-disjoint source-to-sink paths (cycles can be discarded), and any such path collection gives a flow of the same total profit. Thus maximizing profit is exactly minimizing flow cost.

Use the residual graph. Repeatedly find a shortest source-to-sink residual path, augment one unit, and add its cost. Negative original costs are allowed; the reference uses a queue-based Bellman–Ford relaxation. If no augmenting path exists before k units, the requested collection is impossible.

16.2 Correctness proof

The reduction above is value-preserving in both directions. For the algorithm, after an optimal flow of value f , a cheapest residual augmenting path produces an optimal flow of value $f+1$: if a cheaper value- $f+1$ flow existed, subtracting the current flow would decompose into residual cycles and one net source-to-sink augmentation with smaller total residual cost. That contradicts the chosen shortest augmentation (nonnegative-cost residual cycles can be discarded). Induction proves optimality at flow k .

16.3 Complexity and tests

With queue relaxation, the conservative bound is $O(kVE)$ time and $O(V+E)$ memory. Test $k=1$, exactly k outgoing source edges, fewer than k paths, negative profits, and a case where a later residual reverse edge changes an earlier routing choice.

16.4 Full solutions

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct E {
    int to, rev, cap;
    ll cost;
};
int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
int n, m, k;
if (!(cin >> n >> m >> k))
    return 0;
vector<vector<E>> g(n);
auto add = [&](int u, int v, ll c) {
    g[u].push_back({v, (int)g[v].size(), 1, c});
    g[v].push_back({u, (int)g[u].size() - 1, 0, -c});
};
while (m--) {
    int u, v;
    ll p;
    cin >> u >> v >> p;
    add(u - 1, v - 1, -p);
}
ll cost = 0;
int flow = 0;
const ll INF = 4e18;
while (flow < k) {
    vector<ll> d(n, INF);
    vector<int> pv(n), pe(n);
    vector<char> in(n);
    queue<int> q;
    d[0] = 0;
    q.push(0);
    in[0] = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in[u] = 0;
        for (int i = 0; i < (int)g[u].size(); i++) {
            E &e = g[u][i];
            if (e.cap && d[e.to] > d[u] + e.cost) {
                d[e.to] = d[u] + e.cost;
                pv[e.to] = u;
                pe[e.to] = i;
                if (!in[e.to])
                    in[e.to] = 1, q.push(e.to);
            }
        }
    }
    if (d[n - 1] == INF)
        break;
    cost += d[n - 1];
    flow++;
    for (int v = n - 1; v; v = pv[v]) {
        E &e = g[pv[v]][pe[v]];
        e.cap--;
        g[v][e.rev].cap++;
    }
}
}

```

```

if (flow < k)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost << '\n';
}

```

```

from collections import deque
import sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,m,k=d[:3];g=[[[]for _ in range(n)];j=3
    def add(u,v,c):
        g[u].append([v,len(g[v]),1,c]);g[v].append([u,len(g[u])-1,0,-c])
    for _ in range(m):u,v,p=d[j:j+3];j+=3;add(u-1,v-1,-p)
    flow=0;cost=0;inf=10**30
    while flow<k:
        dist=[inf]*n;pv=[-1]*n;pe=[-1]*n;inside=[False]*n;dist[0]=0;q=deque([0]);inside[0]=True
        while q:
            u=q.popleft();inside[u]=False
            for i,e in enumerate(g[u]):
                v,rev,cap,c=e
                if cap and dist[v]>dist[u]+c:
                    dist[v]=dist[u]+c;pv[v]=u;pe[v]=i
                    if not inside[v]:inside[v]=True;q.append(v)
            if dist[-1]==inf:break
            flow+=1;cost+=dist[-1];v=n-1
            while v:
                u=pv[v];i=pe[v];rev=g[u][i][1];g[u][i][2]-=1;g[v][rev][2]+=1;v=u
            print(-cost if flow==k else 'IMPOSSIBLE')
if __name__=="__main__":main()

```

Use a slow mex DFS on tiny DAGs, brute-force congruence search on small moduli, and iterative Fibonacci as independent oracles. Check CRT normalization into $[0, lcm)$, zero Fibonacci, and a DAG vertex with no outgoing edges.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.